

Authenticating the Answer in an Agent Handoff

Single-use, request-bound response authentication and the outcome a caller cannot know

Imran Siddique · OPAQUE Systems · October 1, 2026

Technical report, version 1 · Software experiments

Abstract

One agent appraises a peer, delegates a task to it, and gets an HTTP response back. Appraising the peer before sending says nothing about who wrote that response. A relay can swap the body, replay an answer from another call, or turn an error into a success. We study the strict response-authentication mode of the cA2A runtime, which binds the appraised peer key, the complete request and a fresh request occurrence into a key that can verify exactly one response, once. It authenticates success and denial bodies, binds the status the caller will act on, and reports an unknown outcome whenever verification or transport fails, instead of guessing. On the pinned source, 57 selected tests pass, including loopback HTTP exchanges, and removing the MAC comparison or the single-use consumption makes 4 and 3 of them fail. In 20 rounds of 16 concurrent verifications, exactly one response is accepted per round. With separate caller, relay and peer processes and the connection cut before dispatch, after execution, mid-body, or with the caller killed, the client submits once, reports unknown, and a restarted caller cannot accept the captured response. A control client that resends once on a transport error makes the peer execute one business operation twice. The authentication is for the live caller: the caller can compute the same MAC, so a stored response is not proof of peer authorship to a third party. All results were rerun on October 1, 2026 from a fresh environment and match the earlier records, and the same harness passes on current cA2A main.

Scope of this edition. This is an author-prepared technical report. It has not been peer reviewed, and no independent organization has run the experiments. Every result is software only, on one Windows host operated by the author, over loopback, with software peer assurance. Nothing here shows that a peer key is protected by hardware, and peer-key rotation on attested hardware was not tested. Changes to cA2A after the evaluated commit are listed in section 8.

1. The response is the part nobody signed

A lot of work in agent-to-agent protocols goes into the request direction: who is calling, what they are allowed to delegate, whether the callee is running what it claims. The answer coming back usually gets far less attention. In the A2A protocol [1] the answer is a task result over HTTP. If the caller verified the callee's attestation before sending, it is tempting to treat whatever comes back on that connection as the callee's word.

It is not. TLS may terminate at a load balancer or sidecar. A relay, a retry layer or an error handler may produce the bytes the caller parses. And reusing a business record identifier across calls means a response that only names that identifier can be replayed into a later call for the same record.

The practical questions for anyone running delegated agents are simple. When my agent acts on a peer's answer, did that peer produce it, for this request, this time? And when the answer does not arrive cleanly, what does my agent believe happened?

2. System and threat model

The caller holds an appraised peer public key (X25519) and the assurance level of that appraisal. For each request it generates a fresh X25519 key and a random 32-byte occurrence identifier. The peer holds the private key for the identity the caller appraised. Both derive the response key from their shared secret and the request digest.

The adversary sits on the path. It may change bytes, the HTTP status, the request context, the peer identity, the profile or the response kind. It may replay a response into the same call or another one, send ambiguous JSON, or cut the network after the peer has executed. The caller's process, its local policy and its pending state are trusted. Peer appraisal is an input to this protocol: the response MAC proves that the holder of the appraised key answered, and the appraisal is what says who that is. Either party to the exchange can compute the MAC, which section 4 returns to.

3. Protocol

Request. The caller builds an envelope of `profile` (`ca2a-response-mac-v1`), `peer` (the appraised key), `recipient` (its fresh key), `session` (the random occurrence) and `request` (the complete task). The canonical JSON bytes [2] of that envelope are snapshotted once, so later changes to caller-owned objects cannot change what was bound, and their SHA-256 is the request digest.

Key and statement. HKDF-SHA256 [3] derives the response key from the X25519 shared secret [4], with the request digest as salt and `ca2a/response-mac/v1/key` as context. The response statement holds the profile, request digest, status, kind (provenance for 200, denial for 4xx and 5xx) and body. HMAC-SHA256 [5] authenticates it under a separate `ca2a/response-mac/v1/statement` label.

```
appraised peer + fresh caller key + full request + occurrence
    -> request digest and response key
peer validates context -> executes -> authenticated statement
caller pending state -> consume -> validate -> expose verified body
```

Peer side. The peer validates the envelope shape, the profile and that `peer` is its own key before it executes anything. A malformed context fails before admission. Only after execution does it compute the statement, so an authenticated denial is a real statement from the peer.

Caller side. Verification takes a lock and consumes the pending state first, including on failure, and erases the key. It then checks the deadline (monotonic clock, at most 300 s), parses bounded JSON, requires the exact field set, compares the request digest and the HTTP status the caller observed, verifies the MAC in constant time, checks kind and body shape, and checks the deadline again. Only then does it return the body. The pending object refuses copy, deep copy and pickling, so state cannot be cloned or restored through the supported API.

Parsing. The parser rejects duplicate keys, non-integer numbers, nesting deeper than 64 and input over 1 MiB. Two different readings of the same bytes cannot both be authenticated.

Failure semantics. An authenticated denial is a peer statement; an unsigned error page is not. A failed verification raises `ResponseAuthenticationFailed` with the message "execution outcome is unknown", because a bad MAC does not prove the request failed to execute. The reference client never resubmits after an uncertain response. Losing pending state leaves the old outcome unknown, and a new request is a new occurrence even with the same business record identifier.

4. Properties

Binding. Because the key depends on the request digest, a valid statement from one occurrence fails under any other, under the usual HMAC and HKDF assumptions. Binding the status means a relay cannot turn a 403 into a 200 without breaking the MAC.

Single use. Consumption under a lock means at most one verification attempt per pending state, so concurrent paths in the caller cannot each expose the same accepted response.

Audience. The caller derives the same key, so it can build a statement its own verifier accepts. An exported MAC record therefore cannot prove authorship to an auditor. This is the deniability property that off-the-record messaging chose on purpose [6]. The runtime labels a verified response `audience: live-caller-only`, and that label is accurate. Portable proof of what a peer said needs a signature, which is a different design with different privacy consequences.

5. Evaluation

The subject is cA2A at commit `fc22c644846111c7f375446399e9097a73f93214`, the head of PR 187 [7]. That commit is not on a branch; its squash merge on `main`, `762f329`, has the identical tree (`9e06071`). Every number in this section comes from the rerun of October 1, 2026: Windows 11, Python 3.12.10, a fresh virtual environment built from the commit's hash-pinned `requirements/dev.txt`, and a fresh `git archive` export. The record, logs, JUnit reports and SHA-256 sums are in `experiments/results/rerun-2026-10-01/`. Every outcome is identical to the records of September 23 and 27, 2026.

Selected tests. All **57 tests** in `tests/unit/test_response_authentication.py` pass. They include real loopback HTTP exchanges as well as verifier-level cases: an independent HKDF and HMAC calculation for a committed vector, substituted context fields, malformed JSON, deadlines, replay, peer rotation, strict versus legacy behavior and a response lost after execution. The file is a deliberate selection, not the whole cA2A suite.

Control removals. Replacing the MAC comparison with `if False:` produced **4 failing tests**: two changed-response cases, the substituted response over real HTTP, and the committed vector's negative twins. Keeping pending state and its key for reuse produced **3 failing tests**: one-use, concurrent consumption and consumption on an invalid response. These counts show the selected tests depend on those controls. They are not a mutation-coverage claim.

Concurrency. Twenty rounds of 16 verification attempts on eight worker threads, 320 attempts in all, accepted exactly one response per round.

Caller-built MAC. The caller builds a statement with its own derived key and its verifier accepts it. This is the expected counterexample to reading the MAC as third-party evidence, not an outside attacker defeating the protocol; the experiment reads the private key attribute through a test seam.

6. Cutting the connection

Three kinds of process run on one host. The peer is the real cA2A HTTP server on an ephemeral loopback port, wrapped so that each executed task appends its record identifier to an fsync'd journal. A TCP relay between caller and peer counts submissions, captures the peer's full response, and cuts the caller's connection at a chosen point. Each caller is a new process that writes its own journal entry, then calls `send_task` in strict mode. After each case a new caller process reads the journal, re-appraises the same peer, and offers the captured response to a fresh pending state.

The criterion: the client submits exactly once, reports unknown whenever the response cannot be verified, and a restarted caller cannot turn the captured response into an accepted result.

Relay behavior	Caller outcome	Peer executions	Submissions	Captured response in a new process
Pass through (control row)	accepted	1	1	refused
Cut before dispatch	unknown	0	1	nothing captured
Cut after the peer executed	unknown	1	1	refused
Truncate the response body	unknown	1	1	refused
Hold the response; caller killed	killed, journal says submitted	1	1	refused

All 5 cases met the criterion. The caller sees the same "unknown" when the peer never ran and when it ran once. Only the peer's journal tells them apart, which is exactly why the client must not turn unknown into a retry.

The control changes the client, in an isolated copy, so that `send_task` resends the same request bytes once on `OSError`. Under it, cutting after execution produced 2 peer executions and 2 submissions for one business operation, and cutting before dispatch produced 2 submissions, so 3 of 5 cases met the criterion and 2 failed. The peer accepted the resend because the challenge it issues is stateless by design: an HMAC-protected token that any instance can check, valid until its 60 s expiry, which the cA2A source documents as at-most-once per window [8]. Single submission is therefore a property of the client. A deployment that needs exactly-once execution of a business operation needs deduplication on the peer side, which is what the expired IETF Idempotency-Key draft was written for at the HTTP layer [9].

7. Relation to other work

HTTP Message Signatures lets a response signature cover components of the request through the `req` parameter, and requires the requester to keep those request values until it verifies the response [10]. OSCORE binds each response to its request by putting the request's key identifier and partial IV into the additional authenticated data [11]. The cA2A design uses the same idea at the application layer: the whole canonical request, including a fresh occurrence, feeds the key, and the caller's pending state is consumed on first use. What it adds for agent handoffs is the strict mode a peer cannot downgrade by omitting fields, the bound status and kind, and the explicit unknown outcome. The companion report *Confidentiality Across Agent Handoffs* [12] places this mechanism in a broader composition argument and reports it at the same commit; its 57-test figure is the same selection rerun here and must not be added to this report's count.

8. Later changes on cA2A main

Checked October 1, 2026 against cA2A main 19407fb. The response module, the protocol requirements document and the selected test file are unchanged since the evaluated tree. PR 202 (merged September 25) fails closed on malformed wire input, including a malformed challenge MAC. PR 221 (merged October 1, release 0.4.0) mints caller offers in a caller role and adds a caller proof of possession over the request. Challenges are still stateless. The same harness, run against 19407fb with that commit's own hash-pinned requirements, passes the same 57 tests, fails the same 4 and 3 tests under the two control removals, and reproduces every outcome of sections 5 and 6 (experiments/results/main-19407fb-2026-10-01/). The released `ca2a-runtime 0.4.0` wheel carries `response.py` byte-identical to the pinned source apart from line endings; the harness was not run against the wheel. The evaluated result stays tied to fc22c64.

9. Limits

The response body is readable provenance or denial data; a MAC does not encrypt it, and returning confidential output needs a separate encryption decision. A compromised caller can fabricate its own accepted record, and a compromised peer can truthfully authenticate a wrong answer. The transport experiment ran on one host over loopback and measures no Internet latency or availability. Rotating the peer key on attested hardware and requiring a fresh appraisal before acceptance needs attested peer hardware and was not run. The test oracle and the implementation share some serializers and fixtures; the independent vector reduces that dependency only for the covered ASCII case.

10. Reproduction

From this directory, with a clone of `agentrust-io/ca2a`:

```
git -C <ca2a> fetch origin fc22c644846111c7f375446399e9097a73f93214
git -C <ca2a> checkout --detach fc22c644846111c7f375446399e9097a73f93214
python -m venv <venv>
<venv>/Scripts/python -m pip install --require-hashes -r <ca2a>/requirements/dev.txt
<venv>/Scripts/python -B experiments/run.py --source <ca2a> --output <new directory>
```

If the PR head is ever unavailable, `--commit 762f329aeae589c28219b2cdc2d4a5d6e41407be` exports the identical tree from main. The runner exports the commit with `git archive`, runs the selection, both control removals, the concurrency and caller-MAC experiment, and the transport experiment with its control, and compares every outcome with `experiments/reference-2026-09.json`. Every listener binds to 127.0.0.1. It exits nonzero if a baseline fails, a control survives or a local path leaks into the output.

References

- [1] Agent2Agent (A2A) Protocol Specification, version 1.0.0. Accessed October 1, 2026.
<https://a2a-protocol.org/latest/specification/>
- [2] A. Rundgren, B. Jordan and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, June 2020.
<https://www.rfc-editor.org/rfc/rfc8785.html>
- [3] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)", RFC 5869, May 2010. <https://www.rfc-editor.org/rfc/rfc5869.html>
- [4] A. Langley, M. Hamburg and S. Turner, "Elliptic Curves for Security", RFC 7748, January 2016.
<https://www.rfc-editor.org/rfc/rfc7748.html>
- [5] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication", RFC 2104, February 1997. <https://www.rfc-editor.org/rfc/rfc2104.html>
- [6] N. Borisov, I. Goldberg and E. Brewer, "Off-the-Record Communication, or, Why Not To Use PGP", Workshop on Privacy in the Electronic Society (WPES), 2004, pp. 77 to 84. <https://dl.acm.org/doi/10.1145/1029179.1029200>
- [7] AgentTrust project. cA2A PR 187, Authenticate live handoff responses and preserve timeout diagnostics, merged September 18, 2026; response implementation at commit fc22c64. <https://github.com/agentrust-io/ca2a/pull/187> and https://github.com/agentrust-io/ca2a/blob/762f329aeae589c28219b2cdc2d4a5d6e41407be/src/ca2a_runtime/response.py
- [8] AgentTrust project. cA2A challenge primitive, `src/ca2a_runtime/challenge.py`.
https://github.com/agentrust-io/ca2a/blob/762f329aeae589c28219b2cdc2d4a5d6e41407be/src/ca2a_runtime/challenge.py
- [9] J. Jena and S. Dalal, "The Idempotency-Key HTTP Header Field", Internet-Draft draft-ietf-httpapi-idempotency-key-header-07, expired.
<https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>

- [10] A. Backman, J. Richer and M. Sporny, "HTTP Message Signatures", RFC 9421, February 2024, section 2.4.
<https://www.rfc-editor.org/rfc/rfc9421.html>
- [11] G. Selander, J. Mattsson, F. Palombini and L. Seitz, "Object Security for Constrained RESTful Environments (OSCORE)", RFC 8613, July 2019, sections 5.4 and 7.1. <https://www.rfc-editor.org/rfc/rfc8613.html>
- [12] I. Siddique, "Confidentiality Across Agent Handoffs: Conditions for preserving an inference guarantee through tools and delegation", OPAQUE Systems technical report, version 2, September 30, 2026. DOI 10.5281/zenodo.23074670.
<https://agentrust-io.com/research/confidential-handoffs/>